

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Decentralized Party Governance	Documentation quality	Medium	
Timeline	2026-05-20 through 2026-05-22	Test quality	Medium	
Language	Daml	Total Findings	5	Fixed: 4 Acknowledged: 1
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	0	
Source Code	- DLC-link/dec-party-manager ↗ #c3f578b ↗	Medium severity findings ⓘ	0	
Auditors	- Gereon Mendler Auditing Engineer - Ibrahim Abouzied Auditing Engineer - Darren Jensen Auditing Engineer	Low severity findings ⓘ	3	Fixed: 3
		Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	2	Fixed: 1 Acknowledged: 1

Summary of Findings

This audit covers a decentralized party governance protocol for the Canton Network, where a group of members performs threshold voting on arbitrary action proposals. The decentralized party is bound by the `Rules` contract defining possible behavior. The scope includes several `GovernableAction` implementations to facilitate governance configuration, token custody, and utility onboarding.

While the overall design is sound, this review has identified several low-severity edge case weaknesses, including concerns about audit trails, archival flows, configurations, missing context details and on-chain validations. We recommend addressing these issues to make the protocol more resilient and avoid operational difficulties.

Fix-Review Update 2026-05-22:

The team has addressed all issues, and all fixes align with the recommended changes. `DLC-5` has been acknowledged as intended design to support bootstrapping and development environments. `#S-3` is intended to be handled off-chain, outside the scope of this audit, while `S4` is intentionally not implemented as suggested to minimize the risk surface through generic voting actions. We thank the Bitsafe team for their cooperation and commitment to security.

ID	DESCRIPTION	SEVERITY	STATUS
DLC-1	Permissionless Creation of Unarchivable Confirmations	Low ⓘ	Fixed
DLC-2	Suboptimal InstrumentID Setup	Low ⓘ	Fixed
DLC-3	Governance Execution Audit Trail Lacks Sufficient Proposal Context	Low ⓘ	Fixed
DLC-4	Expired Confirmations Can Still Be Consumed Directly	Informational ⓘ	Fixed
DLC-5	<code>GovernanceRules</code> Permits 1-of-1 Configurations	Informational ⓘ	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

i Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/DLC-link/dec-party-manager/(80b502d)` Files: `daml/*`

Repo: `https://github.com/DLC-link/dec-party-manager`

Included Paths: `daml/governance-core/` , `daml/governance-action-v0/` , `daml/governance-token-custody/` , `daml/governance-utility-onboarding/`

Extensions: `daml`

Files Excluded

Repo: `https://github.com/DLC-link/dec-party-manager/(80b502d)` Files: `daml/dars/*`

Operational Considerations

Voting Records: The vote outcome of governance actions is recorded solely via the `GovernanceExecutionResult` contract created by `GovernanceRules` .

Action Timeout Changes: When the `actionConfirmationTimeout` is changed in the `Rules` contract, it does not apply to existing confirmations, only to those created after the change.

Confirmer Validation Responsibility: Some `GovernableAction` s require the confirming party to do off-chain validations on the proposed action. For example, this applies to proposed transfers and `TransferInstruction` s.

Unreliable Action Inputs: Inputs for `GovernableAction` s may become invalid between proposal time and execution time. For example, this applies to `TransferInstruction` s that may be withdrawn, and `Holding` s that may be used.

Member Set Changes: Members can be added or removed, and the voting threshold can be modified at any point. Any such modification also applies for actions currently being voted on, potentially blocking already executable actions or allowing the execution of previously unsuccessful

proposals.

Fetching Action Results: The action implementation `GovernableAction_Execute` and `executeImpl` have no return type. Many of the interface implementations create templates or execute choices that return templates. These need to be retrieved via the ledger view of the `governanceParty`.

Unsupported Credentialled Tokens: The current `GovernableAction` implementations regarding tokens always pass empty lists for `requirements` and `credentials`. Therefore these actions can only be used to create permissionless tokens.

Action Transparency: Free-text description fields on mint/burn/accept templates are entirely proposer-controlled; members relying on them are trusting the proposer's narration rather than the structured fields. `extraArgsMeta` and meta fields are opaque proposer-supplied metadata, passed verbatim to upstream choices and recorded in the audit trail.

Audit Trail Integrity: `GovernanceExecutionResult` records timestamp only with no diff or description of what changed, no snapshot of the action's actual inputs. Historical reconstruction requires walking the transaction log.

Emergency Controls: No on-chain emergency pause, no partial-quorum revocation, no time-bounded delegations. Revoking any standing authority requires a full multi-sig vote with the configured `actionConfirmationTimeout` for confirmations.

Key Actors And Their Capabilities

Governance Layer

- `governanceParty` — The decentralized multi-sig party that is the on-chain anchor of authority. No single ledger client signs as this party directly; its authority is only obtainable inside `GovernanceRules` choice bodies. Hosted by member participants via `readAs`. Signatory on `GovernanceRules`, `GovernanceExecutionResult`, and (transitively, via `executeImpl`) on every contract produced by a `GovernableAction`.
- `members` (governance committee) — The set of parties tracked on `GovernanceRules.members` who can confirm and execute actions. Each member hosts `governanceParty` on their participant. Voting power is members-only; threshold confirmations from this set are what unlock any privileged operation.
- `additionalProposers` — Optional allowlist of non-member parties tracked on `GovernanceRules.additionalProposers` who may propose actions but cannot confirm or execute. Intended for external admins / federation operators to seed proposals for members to vote on.
- `proposer` — The party submitting a specific `GovernableAction` proposal. Must be a member or an additional proposer per each `GovernanceRules_ConfirmAction` authority check. Signatory of the proposal contract.
- `confirmer` — A member casting a yes-vote, producing a `GovernanceConfirmation` (for domain actions) or `GovernanceSelfConfirmation` (for self-management actions).
- `executor` — Any member who triggers `GovernanceRules_ExecuteConfirmedAction` once threshold confirmations are collected. Recorded on the resulting `GovernanceExecutionResult` for audit.

Utility-Registry Layer

- `operator` — The party controlling the utility-registry app instance. Gatekeeper for provider/user onboarding (accepts `ProviderServiceRequest`, `UserServiceRequest`). Signatory on `ProviderService`, observer on `TransferPreapproval` (utility token), `DelegatedBatchedMarkersProxy`. Load-bearing for the entire registry's operational integrity.
- `provider` — The service provider role within a `ProviderService`. In the BitSafe deployment this is typically `governanceParty`. Signatory on `ProviderService` alongside operator; carries the authority needed to declare instruments and issue/burn tokens.
- `registrar` — The party operating a `RegistrarService` (the contract that performs registration of new instruments).
- `instrumentAdmin` — The admin of a specific instrument under the registry. Referenced in `TransferPreapproval` (utility token) and `InstrumentAllowance` — determines which instruments fall under a given preapproval's scope.

Token-Flow Counterparties

- `holder` — A party holding tokens under a utility-registry instrument. Counterparty on `BurnProposal` (burn offer recipient) and on holder-initiated `BurnRequest` flows.
- `recipient` — Mint counterparty on `MintProposal` (the party who will receive newly-minted tokens upon accepting the offer).
- `issuer` — Conceptually the party empowered to mint/burn an instrument; structurally this is the registry's provider (= `governanceParty`). Issuer-side credential checks reference this role.
- `sender` — Any party initiating a transfer. Not a named field on the governance templates; appears implicitly in the upstream transfer flows (any party can be a sender of CC or utility tokens to the governance party once a `TransferPreapproval` exists).

Findings

DLC-1 Permissionless Creation of Unarchivable Confirmations

• Low ⓘ Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `0edb57f`.

File(s) affected: `daml/governance-core/daml/Governance/Confirmation.daml`, `daml/governance-core/daml/Governance/Rules.daml`

Description: The governance `Rules` contract uses confirmation templates to record votes by members to support certain actions. Members can execute the `GovernanceRules_ConfirmAction` and `GovernanceRules_ConfirmGovernanceAction` choices to vote, but both `GovernanceSelfConfirmation` and `GovernanceConfirmation` do not have the `governanceParty` as a signatory. This allows arbitrary users to create confirmations without following the `Rules` validation/flow. While there's very strong validations in `GovernanceRules_ExecuteConfirmedAction` that block any non-valid confirmations, fake confirmation may be used as spam with `expiresAt` values so high that they can never be archived.

Exploit Scenario:

An attacker scripts the creation of large numbers of `GovernanceConfirmation` or `GovernanceSelfConfirmation` contracts naming a target `governanceParty` with `expiresAt` set decades in the future. The governance party's participant accrues these contracts indefinitely, degrading any tooling that iterates confirmations and risking threshold-tally manipulation if execution-side logic does not strictly filter by an authoritative on-chain membership list.

Recommendation: We suggest listing the `governanceParty` as a signatory on both confirmation templates. This limits the creation process to the intended `Rules` choices and the validations present therein.

DLC-2 Suboptimal InstrumentID Setup

• Low ⓘ Fixed

✓

Update

Marked as "Fixed" by the client.

Addressed in: `8733d06` .

File(s) affected: `daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/SetupUtility.daml`

Description: This implementation of `GovernableAction` is intended to atomically runs the full onboarding chain for a new token in one vote. As part of this process, the `instrumentId` is set to a parameter supplied value, while the `additionalIdentifiers` is always left empty. Best practice is to use both fields for different purposes:

- `instrumentId` : This is typically treated as a globally unique, immutable identifier. To avoid namespace collisions across a decentralized network, this contains a UUID or a highly specific structured string (e.g., `DLC-Link-Utility-Token-v1-UUID`).
- `additionalIdentifiers` : This records standard market symbols. It is typically structured as a list of key-value pairs (e.g., `[("Ticker", "DLC"), ("ISIN", "US1234567890")]`).

Recommendation: Update this action to take and use both a UUID and a ticker, or accept `additionalIdentifiers` directly as a template parameter.

DLC-3 Governance Execution Audit Trail Lacks Sufficient Proposal Context

• Low ⓘ Fixed

i

Update

Marked as "Mitigated" by the client.

Addressed in: `d2309cf` .

The client provided the following explanation:

This commit embeds salient parameter data into the descriptions of nine first-round implementations so the `GovernanceExecutionResult.description` is meaningful without walking the event graph back to the original proposal:

- `AcceptTransferProposal`: includes the `TransferInstruction` contract id
- `TransferProposal`: includes amount, `instrumentId`, and receiver
- `SetupCcPreapprovalProposal`: includes provider party
- `SetupTokenPreapprovalProposal`: includes `instrumentAdmin` party
- `AcceptFreeCredential`: includes the `CredentialOffer` contract id
- `CreateDelegatedBatchedMarkersProxy`: includes operator party
- `CreateProviderServiceRequest`: includes operator and provider parties
- `CreateUserServiceRequest`: includes operator and user parties
- `SetEnableResultContracts`: includes the new `enableResultContracts` value

The companion recommendation (storing `ContractId GovernableAction` on `GovernanceExecutionResult`) is intentionally not adopted: the proposal contract creation event is already in the event graph and reachable by contract id, so an on-chain copy of the id adds no information when the graph is preserved and gives no recovery path when it isn't. We also acknowledge that this enrichment is not enforceable for future external `GovernableAction` implementations written by other teams — our first-round templates are intended as examples of good description discipline.

File(s) affected: `daml/governance-core/daml/Governance/ExecutionResult.daml`

Description: `GovernanceRules_ExecuteConfirmedAction()` creates a `GovernanceExecutionResult` contract as an immutable audit trail after a governable action is executed. The execution result stores metadata such as the `actionLabel`, `description`, `executor`, and `confirmers`.

However, the audit trail currently lacks sufficient information to reliably reconstruct or review the executed proposal.

`GovernanceExecutionResult` does not store the executed `ContractId GovernableAction`, so once `GovernableAction_Execute` archives the proposal contract there is no durable on-chain reference linking the execution result to the original proposal or its confirmations.

Additionally, several `GovernableAction` implementations populate the `description` field with generic values that omit critical execution details. For example, `AcceptTransferProposal` does not include the transferred amount or instrument identifier in its description.

This weakens the governance audit trail and makes it difficult to understand what action was executed or correlate the execution result with the original proposal data.

Recommendation: Ensure each `GovernableActionView.description` contains sufficient action-specific context to understand the executed operation directly from the audit trail.

DLC-4 Expired Confirmations Can Still Be Consumed Directly

• Informational ⓘ Fixed

✓ Update

Marked as "Fixed" by the client.
Addressed in: `59864eb`.

File(s) affected: `daml/governance-core/daml/Governance/Confirmation.daml`

Description: `GovernanceConfirmation` contracts are intended to represent active approvals for a governance proposal until `expiresAt` is reached. The `GovernanceRules_ExecuteConfirmedAction` choice correctly filters out expired confirmations before counting them toward the execution threshold.

However, the `GovernanceConfirmation_Consume` choice itself does not enforce that the confirmation is still valid. If

`GovernanceConfirmation_Consume` is exercised directly, an expired confirmation can still be consumed successfully. The same issue exists in `GovernanceSelfConfirmation_Consume`.

Recommendation: Add an expiry check inside `GovernanceConfirmation_Consume` and `GovernanceSelfConfirmation_Consume`. Revert if `getTime >= expiresAt` before returning the confirmation data.

DLC-5 GovernanceRules Permits 1-of-1 Configurations

• Informational ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Rationale for keeping 1-of-1 legal. Single-signer `GovernanceRules` is intentionally supported for bootstrap and development environments — operators can start with a 1-of-1 configuration and add members through `SelfAction_AddMemberAndSetThreshold` votes as the governance group forms. Hard-coding members `>= 2 && threshold >= 2` would foreclose this bootstrap path on-chain. This is an operational constraint, not an on-chain one. Production deployments are responsible for transitioning to a real quorum (members ≥ 2 , threshold ≥ 2) before they go live. Operators that need an enforced lower bound can wrap the deployment behind their own admission process.

File(s) affected: `daml/governance-core/daml/Governance/Rules.daml`

Description: The `GovernanceRules` template's `ensure` clause accepts `members = {p}` and `threshold = 1` as a valid configuration (the only lower bounds are `not (Set.null members)` and `threshold >= 1`). A `GovernanceRules` contract can therefore be created as a single-signer governance, bypassing the multi-signature requirement entirely — one party can unilaterally confirm and execute every privileged operation the governance party guards.

Recommendation: Consider tightening the `ensure` clause to require `Set.size members >= 2 && threshold >= 2`, and applying the same `>= 2` floors in `validateThresholdForMemberCount` and `RemoveMemberAndSetThreshold` so the single-signer configuration is unrepresentable.

Auditor Suggestions

S1 Improve Onchain Validations

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `f3aa5af`.

File(s) affected: `daml/governance-token-custody/daml/Governance/TokenCustody/SetupCcPreapproval.daml`

Description: Consider using `ensure` statements to validate certain guaranteed properties of valid action inputs, like the receiver of `TransferInstructions`. This can avoid spam of unusable proposals.

For example, `SetupCcPreapprovalProposal` declares `expectedDso : Optional Party` and forwards it into the `TransferPreapprovalProposal` it creates, with no `ensure` clause and no `isSome` check in `executeImpl`. The upstream `Splice.Wallet.TransferPreapproval.TransferPreapprovalProposal` documents this field as *"Must be set"*

Recommendation: Consider failing early at proposal-creation time for inputs that cannot result in valid executions.

S2 Naming Inconsistency for Governance Self-Confirmation Expiry Choice

Fixed



Update

Marked as "Fixed" by the client.
Addressed in: `688c43f`.

File(s) affected: `daml/governance-core/daml/Governance/Rules.daml`

Description: The governance module separates self-management confirmations (`GovernanceSelfConfirmation`) from extensible action confirmations (`GovernanceConfirmation`). Each type has a corresponding expiry choice exposed on `GovernanceRules`.

The `GovernanceRules_ExpireGovernanceConfirmation` choice expires `GovernanceSelfConfirmation` contracts rather than `GovernanceConfirmation` contracts. This naming overlaps with `GovernanceRules_ExpireConfirmation`, which already handles standard `GovernanceConfirmation` expiry. The current naming can cause confusion when integrating or maintaining the governance flows, especially since both confirmation types coexist in the same module.

Recommendation: Rename `GovernanceRules_ExpireGovernanceConfirmation` to `GovernanceRules_ExpireGovernanceSelfConfirmation` to more clearly distinguish it from the standard confirmation expiry flow.

S3 Governable Actions Cannot Return Execution Artifacts

Acknowledged



Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Rationale for deferring. Typed execution artifacts are slated for the planned decman HTTP delegation layer, where governance execution result shape is owned outside the on-chain interface. The auditor offered two paths: (1) extend the `GovernableAction` interface to return typed results, or (2) introduce a standardized execution-artifact contract that implementations populate. Both bake a result-shape decision into the on-chain interface — a decision that the delegation work is intended to make at a different layer.

Introducing either pattern in this batch would foreclose design space in the delegation work. The audit-trail half of this concern is partially addressed by the [DLC-3](#) description-enrichment commit, which embeds salient parameter data (amounts, instrument ids, counterparty parties, related contract ids) into the description fields of nine first-round `GovernableAction` implementations so the on-chain `GovernanceExecutionResult` is meaningful without external lookup.

File(s) affected: `daml/governance-action-v0/daml/Governance/Action.daml`

Description: The `GovernableAction` interface standardizes how governance proposals are executed through `GovernanceRules_ExecuteConfirmedAction()`. Implementations provide their execution logic in `executeImpl()`, which is invoked through `GovernableAction_Execute()` after threshold confirmations are collected.

However, both `GovernableAction_Execute()` and `executeImpl()` have a fixed return type of `Update ()`. This prevents interface implementations from returning execution artifacts created during proposal execution. Several implementations perform actions that create contracts or return useful identifiers, such as `TransferInstruction_Accept()` returning a pending status or sender change contract ID for transfer tracking.

This can make integrations harder, as downstream systems cannot directly retrieve execution outputs from the governance flow and may instead need to scan the ledger for side effects or reconstruct state externally.

Recommendation: Consider extending the `GovernableAction` interface to support typed execution results, or introduce a standardized execution artifact contract that implementations can populate during execution. This would allow governance executions to expose useful downstream metadata without requiring off-chain ledger inspection.

S4 Missing Archive Actions

Acknowledged

i

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

Rationale for not adopting the broad-sweep pattern. A single governance choice with authority to archive any governanceParty-signed contract is too broad. It would let one passed vote sweep arbitrary contracts (existing or future) — a blast radius wider than any single audit-trail entry could meaningfully describe.
We prefer the explicit alternative: when a specific cleanup capability is needed (e.g. archiving stale GovernanceConfirmations after a member is removed), the corresponding template grows a dedicated GovernableAction implementation behind a normal vote. This is more boilerplate but keeps each archive capability narrowly scoped, auditable on its own terms, and visible in the governance action catalog.

Description: The governance has actions to create, but not to archive, many of the templates. Theoretically, arbitrary new implementations of `GovernableAction` could be created and proposed for this purpose. However, for any template the Daml compiler implicitly generates a predefined, consuming choice called `Archive`. Calling `archive cid` is a possible syntactic shorthand to archive any contract where `governanceParty` is the sole signatory.

Recommendation: Create an `AnyTypeCid` archive action that accepts an arbitrary Cid as input and always executes only `archive cid`.

S5 Code Improvements

Fixed

✓

Update
Marked as "Fixed" by the client.
Addressed in: `9b5dba3`.

File(s) affected: `daml/governance-core/daml/Governance/Types.daml`, `daml/governance-core/daml/Governance/Confirmation.daml`

Description: Consider these minor changes to increase code quality:

- Fixed** The `Governance.Types` module appears to be unused and can be removed.
- Fixed** The `Governance.Confirmation` module `GovernanceConfirmation` template uses `assertMsg` when most of the templates use `require`. For consistency consider using a `require` statement.
- Fixed** In the `Governance.Confirmation` module, the `GovernanceConfirmation` template is missing an ensure block to check that `actionLabel` is not empty.

Recommendation: Consider applying the code improvements as described.

Definitions

- High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- Undetermined** – The impact of the issue is uncertain.
- Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.

- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

- Repo: `https://github.com/DLC-link/dec-party-manager`
- `c76...cbc ./daml/governance-action-v0/daml/Governance/Action.daml`
 - `3fb...8b6 ./daml/governance-core/daml/Governance/Confirmation.daml`
 - `eb8...672 ./daml/governance-core/daml/Governance/ExecutionResult.daml`
 - `0be...243 ./daml/governance-core/daml/Governance/GenericVote.daml`
 - `42b...4eb ./daml/governance-core/daml/Governance/Rules.daml`
 - `d02...ac7 ./daml/governance-core/daml/Governance/Types.daml`
 - `b15...3e4 ./daml/governance-token-custody/daml/Governance/TokenCustody/AcceptTransfer.daml`
 - `89c...dc9 ./daml/governance-token-custody/daml/Governance/TokenCustody/SetupCcPreapproval.daml`
 - `3f3...a09 ./daml/governance-token-custody/daml/Governance/TokenCustody/SetupTokenPreapproval.daml`
 - `dc0...9fe ./daml/governance-token-custody/daml/Governance/TokenCustody/TransferProposal.daml`
 - `cba...64a ./daml/governance-utility-onboarding/daml/Governance/TokenIssuance/AcceptBurnRequest.daml`
 - `2ee...c20 ./daml/governance-utility-onboarding/daml/Governance/TokenIssuance/AcceptMintRequest.daml`
 - `814...3c4 ./daml/governance-utility-onboarding/daml/Governance/TokenIssuance/BurnProposal.daml`
 - `f4c...e63 ./daml/governance-utility-onboarding/daml/Governance/TokenIssuance/MintProposal.daml`
 - `41e...e84 ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/CreateDelegatedBatchedMarkersProxy.daml`
 - `504...11f ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/CreateProviderServiceRequest.daml`
 - `59f...04a ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/CreateUserServiceRequest.daml`
 - `859...c71 ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/ProvisionProviderService.daml`
 - `39e...514 ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/SetEnableResultContracts.daml`
 - `435...433 ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/SetProviderAppRewardBeneficiaries.daml`
 - `180...2e4 ./daml/governance-utility-onboarding/daml/Governance/UtilityOnboarding/SetupUtility.daml`

Test Suite Results

Retrieved by running `cargo test` and `dpm test`

```
Running unittests src/lib.rs (target/debug/deps/dec_party_manager-1abe238e968a7f90)

running 111 tests
test auth::validators::common::tests::malformed_token_rejected ... ok
test auth::validators::common::tests::collect_roles_merges_realm_scope_and_flat_roles ... ok
test auth::validators::common::tests::issuer_extraction_handles_url_safe_base64 ... ok
test auth::validators::common::tests::issuer_extraction_strips_trailing_slash ... ok
test auth::validators::common::tests::oidc_issuer_matches_keycloak_shape ... ok
test auth::validators::jwt::tests::resolve_algorithm_falls_back_to_kty_default ... ok
test auth::validators::jwt::tests::resolve_algorithm_rejects_symmetric_and_none ... ok
test config::tests::test_keycloak_defaults_devnet ... ok
test auth::validators::jwt::tests::resolve_algorithm_prefers_jwk_alg ... ok
```



```
test config::tests::test_keycloak_defaults_testnet_mainnet_empty ... ok
test config::tests::test_default_package_config ... ok
test config::tests::test_governance_threshold ... ok
test db::crypto::tests::test_decrypt_plaintext_passthrough ... ok
test db::crypto::tests::test_encrypt_opt_none ... ok
test db::crypto::tests::test_encrypt_decrypt_roundtrip ... ok
test db::sqlite::tests::test_dec_party_owners ... ok
test db::sqlite::tests::test_delete_all_dec_parties ... ok
test db::sqlite::tests::test_dec_party_contracts ... ok
test db::sqlite::tests::test_dec_party_participants ... ok
test db::sqlite::tests::test_get_party_credentials_by_id ... ok
test db::sqlite::tests::test_delete_all_peers ... ok
test db::sqlite::tests::test_get_dec_party_participant_owner_key ... ok
test db::sqlite::tests::test_delete_dec_parties_cascades ... ok
test db::sqlite::tests::test_get_peer_by_public_key ... ok
test db::sqlite::tests::test_get_peer_by_id ... ok
test db::sqlite::tests::test_governance_audit_filters_by_party ... ok
test db::sqlite::tests::test_governance_audit_insert_and_query ... ok
test db::sqlite::tests::test_governance_audit_pagination ... ok
test db::sqlite::tests::test_insert_and_get_dec_parties ... ok
test db::sqlite::tests::test_insert_and_get_peers ... ok
test db::sqlite::tests::test_pending_invitations_roundtrip ... ok
test db::sqlite::tests::test_replace_dec_party_participants_removes_stale ... ok
test db::sqlite::tests::test_replace_preserves_owner_key_when_incoming_is_null ... ok
test db::sqlite::tests::test_upsert_and_get_party_credentials ... ok
test db::sqlite::tests::test_upsert_dec_party_preserves_participant_rows ... ok
test db::sqlite::tests::test_upsert_party_credentials_updates ... ok
test noise::server::tests::data_carrying_commands_preserve_their_payload ... ok
test noise::server::tests::disconnect_never_carries_a_payload ... ok
test noise::tests::load_or_generate_keypair_tightens_permissions ... ok
test noise::tests::parse_chunked_command_metadata_happy_path ... ok
test noise::tests::parse_chunked_command_metadata_too_short ... ok
test noise::tests::parse_chunked_command_metadata_unknown_command_type ... ok
test noise::tests::retry_loop_does_not_retry_on_4xx_bad_status ... ok
test noise::tests::retry_loop_does_not_retry_on_anyhow ... ok
test noise::tests::retry_loop_does_not_retry_on_handshake_failed ... ok
test noise::tests::retry_loop_does_not_retry_on_invalid_message ... ok
test noise::tests::retry_loop_retries_on_5xx_bad_status ... ok
test noise::tests::retry_loop_retries_on_transient_then_succeeds ... ok
test noise::tests::retry_loop_returns_error_on_zero_max_attempts ... ok
test db::sqlite::tests::test_workflow_artifacts_roundtrip_and_cascade ... ok
test noise::tests::retry_loop_returns_terminal_error_after_two_transient_failures ... ok
test noise::tests::retry_loop_succeeds_on_first_attempt ... ok
test db::sqlite::tests::test_workflow_runs_unique_inprogress_per_kind ... ok
test noise::tests::test_message_decoding_too_short ... ok
test noise::tests::save_to_file_creates_0600 ... ok
test noise::tests::test_message_decoding_truncated_payload ... ok
test noise::tests::test_message_encoding_empty ... ok
test noise::tests::test_message_encoding_with_payload ... ok
test noise::tests::test_message_type_conversion ... ok
test noise::tests::test_message_roundtrip ... ok
test participant_id::tests::test_namespace_from_hex ... ok
test participant_id::tests::test_namespace_invalid_length ... ok
test noise::tests::test_parse_flexible_uri ... ok
test participant_id::tests::test_parse ... ok
test participant_id::tests::test_parse_from_file ... ok
test participant_id::tests::test_roundtrip ... ok
test participant_id::tests::test_to_file_format ... ok
test noise::tests::save_to_file_overwrites_existing_with_0600 ... ok
test server::handlers::parties::tests::anyhow_variant_falls_through_to_other ... ok
test server::handlers::parties::tests::peer_error_kind_mapping_known_variants ... ok
test db::sqlite::tests::test_workflow_runs_lifecycle ... ok
test server::handlers::auth::tests::grant_rights_rejects_request_without_principal ... ok
test server::handlers::party_config::tests::discover_member_party_rejects_request_without_principal ...
ok
test server::handlers::auth::tests::grant_rights_mock_mode_returns_canned_rights ... ok
test server::handlers::party_config::tests::save_party_config_rejects_url_change_without_fresh_secret ...
ok
test server::middleware::auth::tests::api_endpoints_require_auth ... ok
test server::middleware::auth::tests::auth_config_is_public_but_auth_status_is_not ... ok
test server::middleware::auth::tests::bearer_scheme_is_case_insensitive ... ok
test server::middleware::auth::tests::bearer_tolerates_extra_whitespace ... ok
```

```
test server::middleware::auth::tests::non_bearer_schemes_rejected ... ok
test server::middleware::auth::tests::put_party_config_bootstrap_allowed_when_empty ... ok
test server::middleware::auth::tests::spa_entry_is_public ... ok
test server::middleware::auth::tests::swagger_is_public ... ok
test server::middleware::auth::tests::bearer_token_passes_with_mock_validator ... ok
test server::middleware::auth::tests::lowercase_bearer_scheme_passes ... ok
test server::middleware::auth::tests::well_known_paths_are_public ... ok
test server::peer_status::tests::bump_advances_timestamp_when_newer ... ok
test server::peer_status::tests::bump_does_not_move_timestamp_backwards ... ok
test server::peer_status::tests::bump_inserts_when_missing ... ok
test server::peer_status::tests::should_probe_returns_false_for_fresh_entry ... ok
test server::peer_status::tests::should_probe_returns_true_for_missing_key ... ok
test server::peer_status::tests::should_probe_returns_true_for_stale_entry ... ok
test server::queries::tests::compare_versions_handles_numeric_segments ... ok
test server::queries::tests::sort_contracts_dedups_by_name_version_template_keeping_latest ... ok
test server::types::tests::action_threshold_rejects_in_add_remove_member ... ok
test server::types::tests::action_threshold_rejects_zero_and_negative ... ok
test server::types::tests::action_timeout_rejects_zero_and_negative ... ok
test server::queries::tests::sort_contracts_by_name_asc_version_desc_created_at_desc ... ok
test server::types::tests::proposal_transfer_rejects_non_positive_amount ... ok
test utils::tests::test_decode_invalid_payload_short ... ok
test server::types::tests::workflow_run_serializes_canton_ids_as_plain_strings ... ok
test utils::tests::test_decode_invalid_payload_truncated ... ok
test utils::tests::test_encode_decode_empty_items ... ok
test utils::tests::test_encode_decode_length_prefixed ... ok
test utils::tests::test_write_and_read_multiple_messages ... ok
test utils::tests::test_write_and_read_single_message ... ok
test workflow::kick::coordinator::tests::split_rejects_trailing_garbage ... ok
test workflow::kick::coordinator::tests::split_rejects_truncated_p2p ... ok
test workflow::kick::coordinator::tests::split_round_trips_for_two_messages ... ok
test server::middleware::auth::tests::jwt_validator_rejects_when_no_trusted_issuer ... ok
test server::middleware::auth::tests::well_known_path_bypasses_auth ... ok
```

test result: ok. 111 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 4.22s

Running tests/governance_workflows.rs (target/debug/deps/governance_workflows-c57bb9743a1d353d)

running 12 tests

```
test common::scenario::tests::ctx_mutations_are_visible_to_later_steps ... ok
test common::scenario::tests::closure_can_capture_string_and_borrow_fixture ... ok
test common::scenario::tests::step_error_wraps_with_scenario_and_step_context ... ok
test common::scenario::tests::steps_execute_in_order ... ok
test common::scenario::tests::then_fail_fasts_on_some_err ... ok
test common::tests::from_env_reports_invalid_port ... ok
test common::tests::from_env_reports_missing_var ... ok
test common::tests::from_env_succeeds_when_all_vars_present ... ok
test governance_workflows_e2e ... ignored, requires running localnet – invoke via integration-
tests/run.sh
test common::scenario::tests::then_times_out_when_probe_only_returns_none ... ok
test common::scenario::tests::polled_then_completes_before_subsequent_step ... ok
test common::scenario::tests::then_retries_until_some_ok ... ok
```

test result: ok. 11 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 4.88s

```
daml/Governance/TestUtils.daml:allocateTestParties: OK
daml/Governance/Test/AuthorizationTest.daml:testUnauthorizedProposerRejectedAtConfirm: OK
daml/Governance/Test/AuthorizationTest.daml:testNonMemberCannotUseGovernanceChoices: OK
daml/Governance/Test/AuthorizationTest.daml:testNonMemberCanCreateProposal: OK
daml/Governance/Test/AuthorizationTest.daml:testMemberProposerAcceptedAtConfirm: OK
daml/Governance/Test/AuthorizationTest.daml:testDuplicateConfirmationsRejected: OK
daml/Governance/Test/AuthorizationTest.daml:testAuthorityFlowsThroughInterface: OK
daml/Governance/Test/AuthorizationTest.daml:testAdditionalProposerProposalCanBeExecutedByMembers: OK
daml/Governance/Test/AuthorizationTest.daml:testAdditionalProposerAcceptedAtConfirm: OK
daml/Governance/Test/ConfirmationTest.daml:testRejectWrongGovernanceParty: OK
daml/Governance/Test/ConfirmationTest.daml:testProposerCanCancelOwnProposal: OK
daml/Governance/Test/ConfirmationTest.daml:testNonProposerCannotProposerCancel: OK
daml/Governance/Test/ConfirmationTest.daml:testNonMemberCannotExpire: OK
daml/Governance/Test/ConfirmationTest.daml:testNonMemberCannotConfirm: OK
daml/Governance/Test/ConfirmationTest.daml:testMemberCanConfirm: OK
daml/Governance/Test/ConfirmationTest.daml:testGovernancePartyCanStillCancel: OK
daml/Governance/Test/ConfirmationTest.daml:testExpireStaleConfirmation: OK
daml/Governance/Test/ConfirmationTest.daml:testConfirmerCanCancel: OK
```

daml/Governance/Test/ConfirmationTest.daml:testCannotExpireValidConfirmation: OK
daml/Governance/Test/ExecutionTest.daml:testNonMemberCannotExecute: OK
daml/Governance/Test/ExecutionTest.daml:testFailingActionExecution: OK
daml/Governance/Test/ExecutionTest.daml:testExpiredConfirmationsDontCount: OK
daml/Governance/Test/ExecutionTest.daml:testExecutionWithExtraConfirmations: OK
daml/Governance/Test/ExecutionTest.daml:testConfirmationsCannotBeMixed: OK
daml/Governance/Test/ExecutionTest.daml:given_governance_setup: OK
daml/Governance/Test/ExecutionTest.daml:testInsufficientConfirmations: OK
daml/Governance/Test/ExecutionTest.daml:test_when_proposal_executed_then_action_and_result_created: OK
daml/Governance/Test/GenericVoteTest.daml:given_governance_setup: OK
daml/Governance/Test/GenericVoteTest.daml:test_when_all_members_confirm_then_vote_passes: OK
daml/Governance/Test/GenericVoteTest.daml:test_when_multiple_votes_then_results_are_independent: OK
daml/Governance/Test/GenericVoteTest.daml:test_when_vote_executed_then_result_records_description: OK
daml/Governance/Test/RulesTest.daml:testRejectZeroThresholdOnCreate: OK
daml/Governance/Test/RulesTest.daml:testRejectThresholdExceedingMembersOnCreate: OK
daml/Governance/Test/RulesTest.daml:testRejectShortTimeoutOnCreate: OK
daml/Governance/Test/RulesTest.daml:testRejectNegativeThresholdOnCreate: OK
daml/Governance/Test/RulesTest.daml:testRejectEmptyMembers: OK
daml/Governance/Test/RulesTest.daml:testGovernanceRulesCreation: OK
daml/Governance/Test/RulesTest.daml:testCustomConfiguration: OK
daml/Governance/Test/RulesTest.daml:testAllowMinimumThresholdOnCreate: OK
daml/Governance/Test/SelfActionTest.daml:testStrangerCannotConfirmSelfAction: OK
daml/Governance/Test/SelfActionTest.daml:testSetTimeout: OK
daml/Governance/Test/SelfActionTest.daml:testSetThreshold: OK
daml/Governance/Test/SelfActionTest.daml:testRemoveMember: OK
daml/Governance/Test/SelfActionTest.daml:testRemoveAdditionalProposer: OK
daml/Governance/Test/SelfActionTest.daml:testRejectThresholdLessThanOne: OK
daml/Governance/Test/SelfActionTest.daml:testRejectThresholdExceedingMembers: OK
daml/Governance/Test/SelfActionTest.daml:testRejectShortTimeout: OK
daml/Governance/Test/SelfActionTest.daml:testRejectAddMemberWhoIsAdditionalProposer: OK
daml/Governance/Test/SelfActionTest.daml:testNonMemberCannotExecuteSelfAction: OK
daml/Governance/Test/SelfActionTest.daml:testNonMemberCannotConfirmSelfAction: OK
daml/Governance/Test/SelfActionTest.daml:testMismatchedSelfAction: OK
daml/Governance/Test/SelfActionTest.daml:testInsufficientSelfConfirmations: OK
daml/Governance/Test/SelfActionTest.daml:testAdditionalProposerSeedsSelfAction: OK
daml/Governance/Test/SelfActionTest.daml:testAdditionalProposerAloneCannotExecute: OK
daml/Governance/Test/SelfActionTest.daml:testAddMember: OK
daml/Governance/Test/SelfActionTest.daml:testAddAdditionalProposer: OK

daml/Governance/TokenCustody/TestUtils.daml:allocateTokenTestParties: OK
daml/Governance/TokenCustody/Test/AcceptTransferTest.daml:given_governance_with_token_setup: OK
daml/Governance/TokenCustody/Test/AcceptTransferTest.daml:test_when_transfer_accepted_then_governance_receives_holdings: OK
daml/Governance/TokenCustody/Test/AcceptTransferTest.daml:test_when_withdrawn_offer_accepted_then_execution_fails: OK
daml/Governance/TokenCustody/Test/AcceptTransferTest.daml:test_when_non_member_confirms_then_confirmation_rejected: OK
daml/Governance/TokenCustody/Test/OfferTransferTest.daml:given_governance_with_token_setup: OK
daml/Governance/TokenCustody/Test/OfferTransferTest.daml:test_when_zero_amount_then_proposal_rejected: OK
daml/Governance/TokenCustody/Test/OfferTransferTest.daml:test_when_insufficient_confirmations_then_execution_rejected: OK
daml/Governance/TokenCustody/Test/OfferTransferTest.daml:test_when_non_member_confirms_then_confirmation_rejected: OK
daml/Governance/TokenCustody/Test/OfferTransferTest.daml:test_when_transfer_offered_then_offer_created: OK
daml/Governance/TokenCustody/Test/SendTokensTest.daml:given_governance_with_token_setup: OK
daml/Governance/TokenCustody/Test/SendTokensTest.daml:test_when_zero_amount_then_proposal_rejected: OK
daml/Governance/TokenCustody/Test/SendTokensTest.daml:test_when_insufficient_confirmations_then_execution_rejected: OK
daml/Governance/TokenCustody/Test/SendTokensTest.daml:test_when_non_member_confirms_then_confirmation_rejected: OK
daml/Governance/TokenCustody/Test/SendTokensTest.daml:test_when_tokens_sent_then_outsider_receives_holdings: OK
daml/Governance/TokenCustody/Test/SetupCcPreapprovalTest.daml:given_governance_setup: OK
daml/Governance/TokenCustody/Test/SetupCcPreapprovalTest.daml:test_when_cc_preapproval_proposed_then_proposal_created: OK
daml/Governance/TokenCustody/Test/SetupCcPreapprovalTest.daml:test_when_no_dso_then_preapproval_created: OK
daml/Governance/TokenCustody/Test/SetupCcPreapprovalTest.daml:test_when_non_member_confirms_then_confirmation_rejected: OK
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:given_governance_with_token_setup: OK
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:given_governance: OK


```
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:test_when_empty_allowances_then_preapproval_created: OK
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:test_when_token_preapproval_proposed_then_preapproval_created: OK
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:test_when_preapproval_setup_then_tokens_receivable: OK
daml/Governance/TokenCustody/Test/SetupTokenPreapprovalTest.daml:test_when_non_member_confirms_then_confirmation_rejected: OK

daml/Governance/UtilityOnboarding/TestUtils.daml:allocateOnboardingTestParties: OK
daml/Governance/UtilityOnboarding/Test/AcceptBurnRequestTest.daml:given_burn_request_exists: OK
daml/Governance/UtilityOnboarding/Test/AcceptBurnRequestTest.daml:test_when_accept_burn_request_then_holding_burned: OK
daml/Governance/UtilityOnboarding/Test/AcceptBurnRequestTest.daml:test_when_outsider_confirms_accept_burn_then_rejected: OK
daml/Governance/UtilityOnboarding/Test/AcceptMintRequestTest.daml:given_mint_request_exists: OK
daml/Governance/UtilityOnboarding/Test/AcceptMintRequestTest.daml:test_when_accept_mint_request_then_holding_minted: OK
daml/Governance/UtilityOnboarding/Test/AcceptMintRequestTest.daml:test_when_outsider_confirms_accept_mint_then_rejected: OK
daml/Governance/UtilityOnboarding/Test/BurnProposalTest.daml:given_setup_done: OK
daml/Governance/UtilityOnboarding/Test/BurnProposalTest.daml:test_when_burn_proposed_then_burn_offer_created: OK
daml/Governance/UtilityOnboarding/Test/BurnProposalTest.daml:test_when_zero_amount_burn_then_create_rejected: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:given_setup_done: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:given_governance: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_set_provider_app_reward_beneficiaries: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_create_user_service_request: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_create_provider_service_request: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_create_delegated_batched_markers_proxy: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_set_enable_result_contracts: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:given_governance_with_operator_as_additional_proposer: OK
daml/Governance/UtilityOnboarding/Test/GranularActionsTest.daml:test_provision_provider_service: OK
daml/Governance/UtilityOnboarding/Test/MintProposalTest.daml:given_setup_done: OK
daml/Governance/UtilityOnboarding/Test/MintProposalTest.daml:test_when_mint_proposed_then_mint_offer_created: OK
daml/Governance/UtilityOnboarding/Test/MintProposalTest.daml:test_when_zero_amount_then_create_rejected: OK
daml/Governance/UtilityOnboarding/Test/SetupUtilityTest.daml:given_governance: OK
daml/Governance/UtilityOnboarding/Test/SetupUtilityTest.daml:test_when_non_member_confirms_then_rejected: OK
daml/Governance/UtilityOnboarding/Test/SetupUtilityTest.daml:test_when_setup_without_flags_then_no_optional_artifacts: OK
daml/Governance/UtilityOnboarding/Test/SetupUtilityTest.daml:test_when_setup_proposed_then_artifacts_created: OK
```

Changelog

- 2026-05-22 - Initial report
- 2026-06-05 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

